BASI DI DATI

Caratteristiche evolute dei DBMS relazionali

Daniele Riboni · Università degli Studi di Cagliari
Dipartimento di Matematica e Informatica

Slide tratte in parte dal materiale didattico del libro: Atzeni, Ceri, Fraternali, Paraboschi, Torlone. Basi di dati. Quarta edizione. McGraw-Hill, 2014

BASI DI DATI

Argomenti

1. Transazioni e proprietà ACID
2. Basi di dati attive e trigger
3. Autorizzazioni e controllo degli accessi

BASI DI DATI

Transazioni e proprietà ACID

TRANSAZIONI

Una funzionalità fondamentale dei DBMS

Transazione: **Sequenza** di operazioni da considerare **indivisibile** (“atomica”), corretta anche in presenza di **concorrenza** e con effetti **definitivi**



TRANSAZIONI

Esempio di transazione

Un bonifico di 100 euro dal conto di Tim al conto di Tom:

```
Inizio_transazione;
Sottrai 100 euro dal saldo del conto di Tim
Aggiungi 100 euro al saldo del conto di Tom
Fine transazione
```

TRANSAZIONI IN SQL

Terminano con Commit o Rollback

Conclusione di una transazione:

`commit`

Le operazioni specificate dall'inizio della transazione vengono eseguite sulla base di dati

`rollback`

Si rinuncia all'esecuzione delle operazioni specificate dopo l'inizio della transazione

I confini della transazioni

Una transazione inizia al primo comando SQL dopo la “connessione” di un utente alla base di dati, oppure alla conclusione di una precedente transazione

Lo standard indica anche un comando `start transaction`, non obbligatorio, e quindi non previsto in molti sistemi

Esempio di una transazione in SQL

```
start transaction;
update ContoCorrente
  set Saldo = Saldo - 10
  where NumeroConto = 123;
update ContoCorrente
  set Saldo = Saldo + 10
  where NumeroConto = 456;
commit work;
```

E in PostgreSQL / PgAdmin?

- **Autocommit attivo (default in pgAdmin)**: ogni comando eseguito singolarmente apre e chiude la propria transazione
- **Più comandi eseguiti insieme in un solo "Execute"** (separati da ; nello stesso batch): PostgreSQL li tratta come un'unica transazione implicita, anche con autocommit attivo
- **Autocommit disattivato** (o BEGIN esplicito): la transazione resta aperta finché non si esegue manualmente COMMIT o ROLLBACK

E se mi collego a PostgreSQL da programma?

Dipende da quale libreria uso. Per esempio, in Python:

- **psycopg2 (Python)**: autocommit **disattivato di default** — comportamento opposto a pgAdmin
  - Prima **cursor.execute()**: apre implicitamente una transazione (BEGIN invisibile)
  - **Comandi successivi** restano parte della **stessa transazione** finché non si chiama esplicitamente conn.commit() o conn.rollback()

- **Se il programma termina o va in errore prima del commit**: tutte le modifiche vengono perse

- **Per avere comportamento autocommit** (come PgAdmin)**:** conn.autocommit = True

ESEMPIO DI TRANSAZIONE

Anna compra 2 unità del prodotto P1 a 25 € l'una

CASO 1 · TUTTE LE OPERAZIONI VANNO A BUON FINE



CASO 2 · ACQUISTO CONCORRENTE: LA SCORTA DIVENTA NEGATIVA



ESEMPIO CON E SENZA TRANSAZIONE

Marco supera Basi di Dati (9 CFU) con voto 28

CASO 1 · CON TRANSAZIONE, GUASTO DOPO IL PASSO 2



CASO 2 · SENZA TRANSAZIONE, GUASTO DOPO IL PASSO 2





TRANSAZIONI

Proprietà ACID (“acide”)

A
Atomicità

C
Consistenza

I
Isolamento

D
Durability (persistenza)

PROPRIETÀ ACID

Atomicità

La sequenza di operazioni sulla base di dati viene eseguita **per intero o per niente**:

Trasferimento di fondi da un conto A a un conto B: o si fanno il prelevamento da A e il versamento su B, o nessuno dei due

Se la transazione non viene completata correttamente (es. per guasto, violazione di vincoli, o su richiesta dell'utente) il DBMS annulla tutta la transazione e riporta il DB allo stato iniziale

PROPRIETÀ ACID – ATOMICITA’

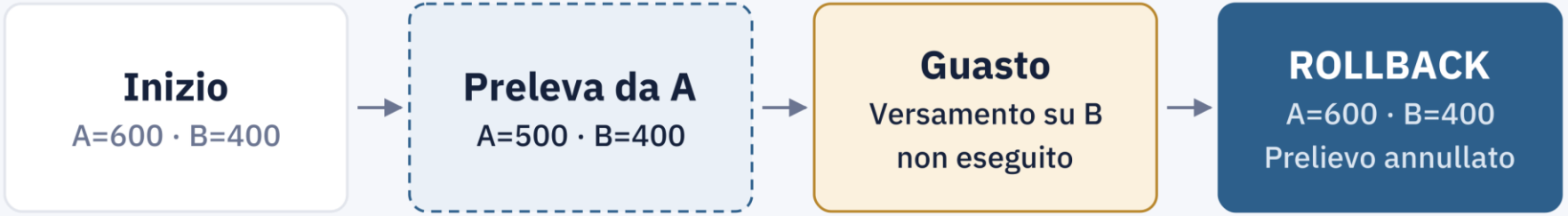
Esempio bonifico

Trasferimento di 100 € dal conto A al conto B

CASO 1 · TUTTE LE OPERAZIONI VANNO A BUON FINE



CASO 2 · GUASTO A METÀ, NON VIENE APPLICATO NIENTE



Stato stabile · Operazione in corso · Guasto · Esito finale

PROPRIETÀ ACID

Consistenza

Al termine dell'esecuzione di una transazione, i **vincoli di integrità** devono essere soddisfatti

Durante l'esecuzione ci possono essere violazioni, ma se restano alla fine allora la transazione deve essere annullata per intero

PROPRIETÀ ACID - CONSISTENZA

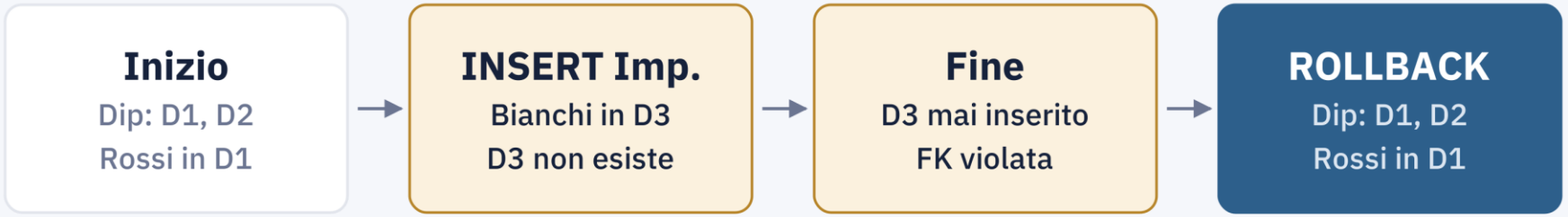
Esempio impiegati e dipartimenti

Vincolo: Impiegato.dip → Dipartimento.codice

CASO 1 · ALLA FINE IL VINCOLO È RISPETTATO



CASO 2 · ALLA FINE IL VINCOLO È ANCORA VIOLATO



Vincolo rispettato Vincolo violato Esito finale

PROPRIETÀ ACID

Isolamento

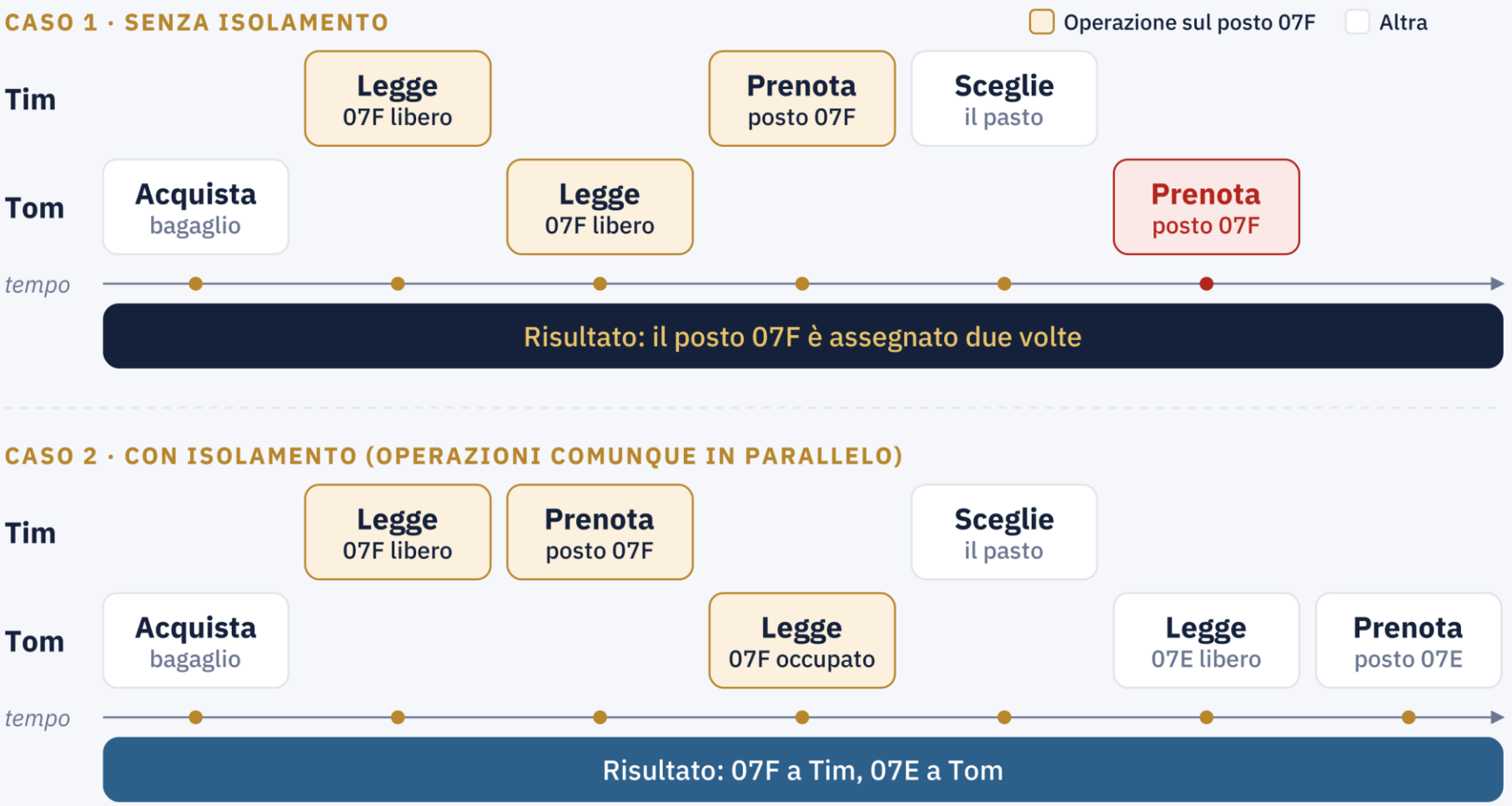
L'effetto di transazioni concorrenti deve essere coerente (nel caso Migliore **equivalente all'esecuzione seriale**)

Se due assegni emessi sullo stesso conto corrente vengono incassati contemporaneamente, si deve evitare di trascurarne uno

L'isolamento mira a garantire che l'esecuzione concorrente delle transazioni produca risultati coerenti. Il livello più forte, SERIALIZABLE, garantisce un risultato equivalente a una qualche esecuzione seriale delle transazioni. Non tutti i livelli di isolamento la garantiscono.

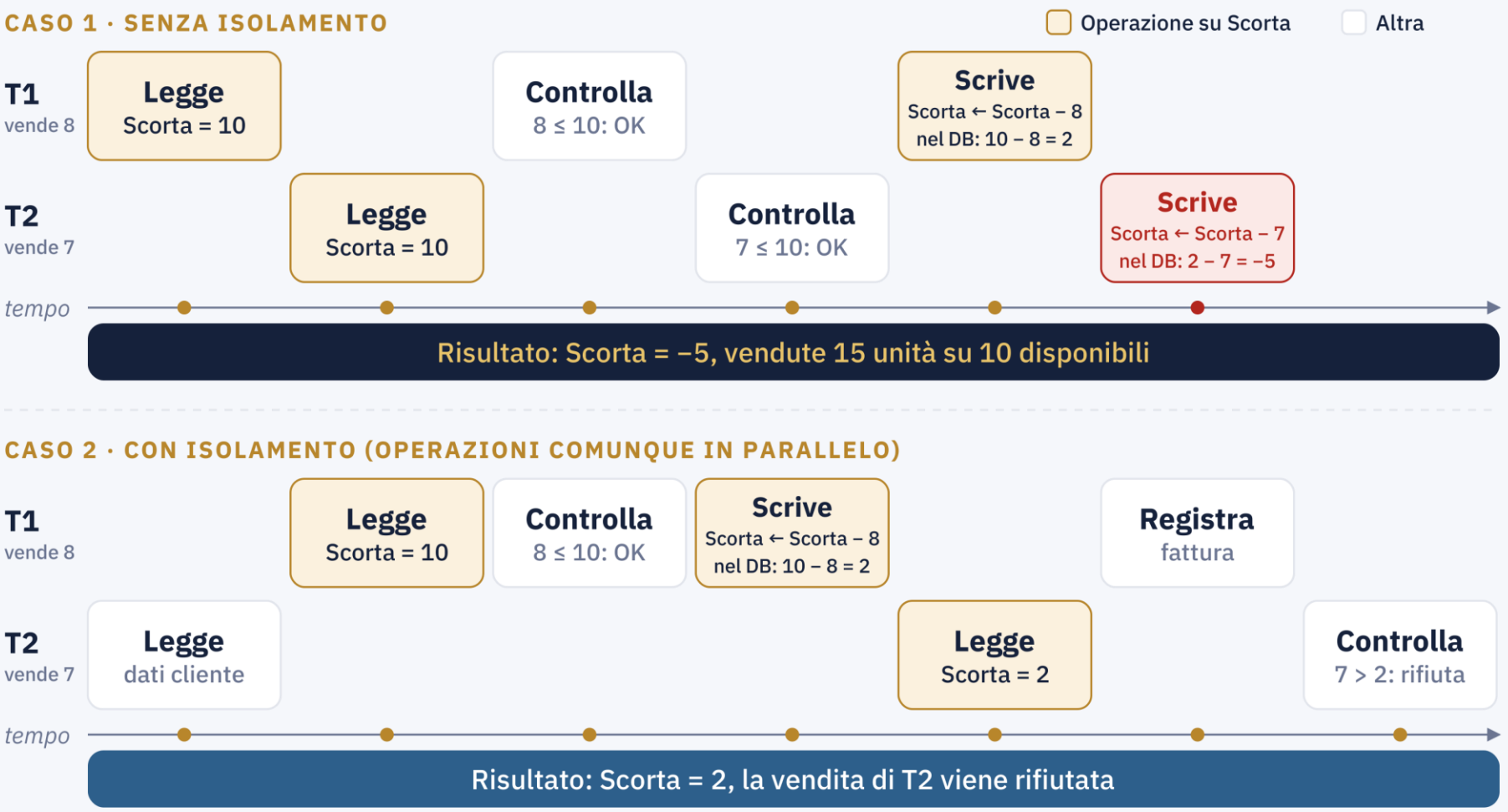
PROPRIETÀ ACID - ISOLAMENTO

Esempio prenotazione posto su di un volo



PROPRIETÀ ACID - ISOLAMENTO

Esempio vendita prodotti



PROPRIETÀ ACID

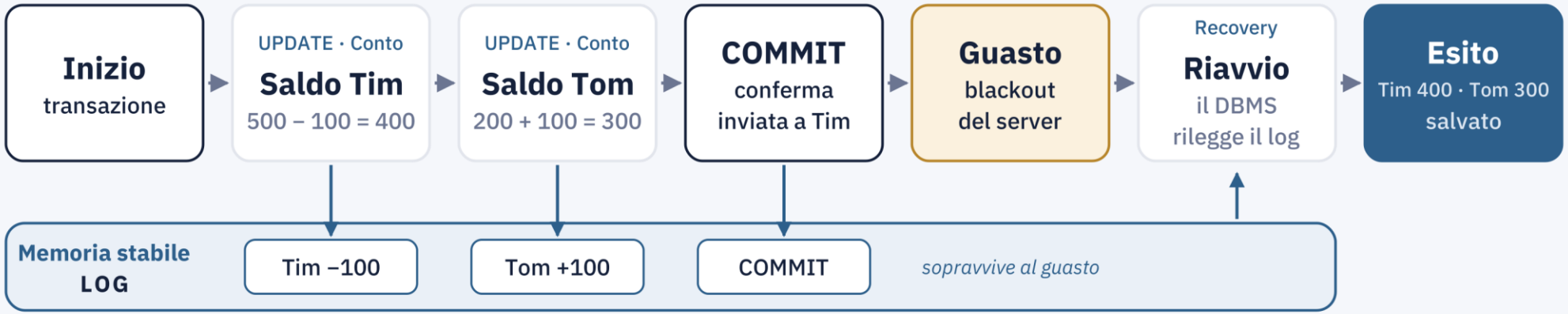
Durability (persistenza)

La conclusione positiva di una transazione corrisponde a un impegno (`commit`) a mantenere traccia del risultato in modo definitivo, anche in presenza di guasti

PROPRIETÀ ACID – ESEMPIO DI DURABILITY

Bonifico di 100 € dal conto di Tim al conto di Tom

CASO 1 · GUASTO DOPO IL COMMIT



CASO 2 · GUASTO PRIMA DEL COMMIT



Non c'è certezza che il secondo UPDATE sia stato registrato nel log prima del guasto

TRANSAZIONI

Supporto delle proprietà ACID

I DBMS relazionali garantiscono le proprietà ACID (in maniera variabile e configurabile), ma garantirle non è semplice: ha un impatto negativo sull'efficienza del DBMS

NoSQL

Nuovi DBMS non-relazionali: in generale supportano versioni deboli di ACIDità per essere più efficienti

NewSQL

DBMS relazionali di nuova generazione: cercano di conciliare ACID ed efficienza

BASI DI DATI

Basi di dati attive e trigger

BASI DI DATI ATTIVE

Rispondono automaticamente a determinati eventi

Una base di dati che contiene regole attive (chiamate `trigger`)

Paradigma: Evento — Condizione — Azione

1. Quando un evento si verifica
2. Se la condizione è vera
3. Allora l'azione è eseguita

Evento, condizione, azione

1. **Evento** — normalmente una modifica dello stato del database (insert, delete, update); quando accade l'evento, il trigger è attivato
2. **Condizione** — un predicato che identifica se l'azione del trigger deve essere eseguita
3. **Azione** — una sequenza di comandi SQL

BASI DI DATI ATTIVE

Il concetto di trigger

Questo modello consente computazioni reattive

Non è il solo tipo di regole: azioni compensative nei vincoli di integrità

Vantaggio (rispetto a implementare le regole esternamente): **la logica dei dati è separata dalla logica delle applicazioni**

SINTASSI

I trigger in SQL

Ogni trigger è caratterizzato da:

- Nome
- Target (tabella controllata)
- Modalità (before o after)
- Evento (insert, delete o update)
- Granularità (statement-level o row-level)
- Azione

Diversi DBMS usano sintassi diverse per i trigger: useremo una pseudo-sintassi semplificata

SINTASSI SQL

Il comando “create trigger”

```
create trigger NomeTrigger
  { before | after } { insert | delete | update [of Colonna] }
  on TabellaTarget
  [referencing
     {[old table [as] VarTabellaOld] [new table [as] VarTabellaNew]} |
     {[old [row] [as] VarTuplaOld] [new [row] [as] VarTuplaNew]}]
  [for each { row | statement }]  [when Condizione]
  Statement Procedurale SQL
```

GRANULARITÀ DEGLI EVENTI

A livello di intero comando

STATEMENT-LEVEL (DI DEFAULT)

- Il trigger viene considerato e possibilmente eseguito solo una volta per ogni statement (comando) che lo ha attivato, indipendentemente dal numero di tuple modificate

- In linea con SQL (set-oriented)

GRANULARITÀ DEGLI EVENTI

A livello di singola riga

ROW-LEVEL

- Il trigger viene considerato e possibilmente eseguito una volta per ogni riga modificata

GRANULARITÀ DEGLI EVENTI

Variabili e granularità

Se granularità = row-level

- `old`: tupla nello stato precedente la modifica
- `new`: tupla nello stato prodotto dalla modifica

Se granularità = statement-level

- `old_table`: stato precedente della porzione di tabella interessata
- `new_table`: stato prodotto dalla modifica

MODALITÀ DEI TRIGGER

Modalità BEFORE

Il trigger è considerato e possibilmente eseguito **prima** della modifica

- I trigger before non possono modificare lo stato del database: possono al più condizionare i valori “new”
- Normalmente questa modalità è usata quando si vuole verificare una modifica prima che essa avvenga e “modificare la modifica”

MODALITÀ BEFORE

Esempio di trigger in modalità BEFORE

1. “Conditioner” (agisce prima dell'update e della verifica di integrità)

```
create trigger LimitaAumenti
before update of Salario on Impiegato
for each row
when (New.Salario > Old.Salario * 1.2)
  set New.Salario = Old.Salario * 1.2
```

Vantaggio: *il controllo avviene prima dell'operazione, evitando scritture inutili se la condizione è vera; se però la condizione è vera ma l'operazione viene poi rifiutata, l'azione del trigger è comunque stata eseguita*

MODALITÀ DEI TRIGGER

Modalità AFTER

Il trigger è considerato ed eseguito **dopo** l'evento

È la modalità più comune

MODALITÀ AFTER

Esempio di trigger in modalità AFTER

2. “Re-installer” (agisce dopo l'update)

```
create trigger LimitaAumenti
after update of Salario on Impiegato
for each row
when (New.Salario > Old.Salario * 1.2)
  update Impiegato
  set Salario = Old.Salario * 1.2
  where ImpNum = new.ImpNum
```

Vantaggio: *se l'operazione non va a buon fine, si evita di valutare il trigger; se invece va a buon fine, occorre sovrascrivere quanto già scritto*

GRANULARITÀ

Esempio di trigger row-level

```
create trigger AccountMonitor
after update on Account
for each row
when new.Totale < old.Totale
  insert values
    (new.NumeroConto, old.Totale - new.Totale, now())
  into Prelievi
```

GRANULARITÀ

Esempio di trigger statement-level

```
create trigger ArchiviaFattureCancellate
after delete on Fattura
  insert into FattureCancellate
    (select * from old_table)
```

ESEMPI

Creazione di trigger

Nei prossimi esempi usiamo due semplici tabelle:

Prodotti

(CodProdotto, Nome, Prezzo, Scorta)

Vendite

(CodVendita, CodProdotto, Quantita, DataVendita)

Ogni vendita registra la quantità venduta di un prodotto; Scorta è la giacenza disponibile

ESEMPI

Creazione di trigger AFTER row-level

AFTER · ROW-LEVEL

Quesito: ad ogni inserimento in Vendite, decrementa la Scorta del prodotto venduto della quantità venduta.

Soluzione:

```
create trigger AggiornaScorta
after insert on Vendite
for each row
  update Prodotti
  set Scorta = Scorta - New.Quantita
  where CodProdotto = New.CodProdotto
```

ESEMPI

Creazione di trigger BEFORE row-level

BEFORE · ROW-LEVEL

Quesito: prima di ogni inserimento in Vendite, annulla l'operazione se la quantità richiesta supera la Scorta disponibile del prodotto.

Soluzione:

```
create trigger VerificaScorta
before insert on Vendite
for each row
when (New.Quantita > (select Scorta from Prodotti
  where CodProdotto = New.CodProdotto))
  rollback
```

ESEMPI

Creazione di trigger AFTER statement-level

AFTER · STATEMENT-LEVEL

Quesito: dopo ogni modifica (anche su più righe) a Prodotti, annulla l'operazione se un prodotto risulta con Scorta negativa.

Soluzione:

```
create trigger VerificaScorte
after update on Prodotti
  when exists
    (select * from new_table where Scorta < 0)
  rollback
```

ESEMPI

Creazione di trigger BEFORE statement-level

BEFORE · STATEMENT-LEVEL

Quesito: prima di un inserimento massivo in Vendite, blocca l'intera operazione se la quantità totale delle nuove vendite supera 1000 unità (un controllo sull'intero lotto, non sulla singola riga: per questo serve lo statement-level).

Soluzione:

```
create trigger LimitaOrdineTotale
before insert on Vendite
  when (select sum(Quantita) from new_table) > 1000
  rollback
```

CONFRONTO

BEFORE e AFTER a confronto

Lo stesso evento — un insert in Vendite — gestito in due momenti diversi:

PRIMA · BEFORE

`VerificaScorta`

Condiziona i valori proposti e può impedire la scrittura: la vendita non viene mai registrata se la Scorta non basta.

DOPO · AFTER

`AggiornaScorta`

Agisce sui dati già scritti: la Scorta viene aggiornata subito dopo che la vendita è stata registrata.

BEFORE impedisce o corregge prima che la modifica avvenga; **AFTER** reagisce a una modifica già avvenuta.

CONFRONTO

Due trigger AFTER, in cascata

AggiornaScorta e VerificaScorte sono entrambi after, ma su tabelle ed eventi diversi:

AFTER · ROW-LEVEL (SU VENDITE)

`AggiornaScorta`

Dopo ogni insert in Vendite, riga per riga, aggiorna (con un update) la Scorta del prodotto venduto in Prodotti.

AFTER · STATEMENT-LEVEL (SU PRODOTTI)

`VerificaScorte`

Dopo ogni update su Prodotti — anche quelli generati da AggiornaScorta — controlla se una Scorta è negativa e nel caso fa rollback.

L'update eseguito da AggiornaScorta è a sua volta un evento: **attiva VerificaScorte**, pur non essendo mai collegato direttamente a un insert in Vendite — una catena di trigger (cascading trigger).

STRATEGIE

Due strategie: quale conviene?

Entrambe evitano una Scorta negativa, ma non sono equivalenti:

BEFORE

`VerificaScorta`

Come agisce: blocca l'insert se eccede la Scorta.
Limite: una transazione concorrente può bypassare il controllo (vedi esempio “Anna”).

AFTER

`AggiornaScorta`

Come agisce: scrive sempre; poi VerificaScorte verifica e fa rollback se serve.

Punto di forza: controlla lo stato realmente scritto, anche sotto concorrenza.

Sono entrambe valide, ma non bastano singolarmente: **before** è efficiente ma non robusto sotto concorrenza; **after** è robusto ma scrive (ed eventualmente disfa) anche i casi evitabili. La soluzione più solida le combina entrambe.

ESECUZIONE DEI TRIGGER

Trigger in conflitto

Quando vi sono più trigger associati allo stesso evento (in conflitto), vengono eseguiti come segue:

1. Per primi i before triggers
2. Poi viene eseguita la modifica
3. Infine sono eseguiti gli after triggers

Quando vari trigger appartengono alla stessa categoria, l'ordine di esecuzione è definito in base al loro timestamp di creazione, a seconda del DBMS: alcuni danno priorità più alta ai trigger più vecchi, altri si basano sull’ordine alfabetico

ESECUZIONE DEI TRIGGER

Terminazione

L'azione di un trigger può produrre eventi che attivano altri trigger: si possono generare dei **cicli infiniti**

Possibili esiti:

- L'esecuzione termina correttamente
- L'esecuzione termina in errore quando si raggiunge una data profondità di ciclo
- Viene fatto un rollback: si ripristina lo stato precedente l'attivazione dei trigger

ESECUZIONE DEI TRIGGER

Analisi della terminazione

Si usa una rappresentazione delle regole detta **grafo di triggering**:

- Un nodo per ogni trigger
- Un arco dal nodo ti al nodo tj se l'esecuzione dell'azione di ti può attivare il trigger tj

Se il grafo è aciclico, l'esecuzione termina sempre

Se il grafo ha cicli, esso può avere problemi di terminazione

ANALISI DELLA TERMINAZIONE

Esempio con due trigger

T1

```
create trigger AggiornaContributi
after update of Stipendio on Impiegato
  update Impiegato
  set Contributi = Stipendio * 0.8
  where Matr in
    (select Matr from new_table)
```

(mi permette di garantire un vincolo di tupla)

ANALISI DELLA TERMINAZIONE

Esempio con due trigger

T2

```
create trigger ControllaSogliaBudget
after update on Impiegato
when 50000 < (select (New.Stipendio + New.Contributi)
              from Impiegato)
  update Impiegato
  set Stipendio = 0.9 * Stipendio
```

ANALISI DELLA TERMINAZIONE

Grafo di triggering corrispondente



Ci sono due cicli, ma il sistema termina sempre

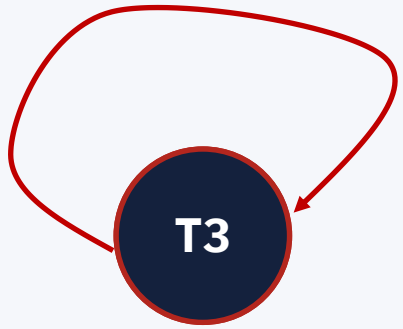
Sarebbe non terminante se in T2 il fattore di moltiplicazione fosse maggiore di 1

ANALISI DELLA TERMINAZIONE

Esempio di non terminazione

T3

```
create trigger ControllaStipendio
after update on Impiegato
when New.Stipendio < 50000
  update Impiegato
  set Stipendio = 0.9 * Stipendio
```



Ogni esecuzione riduce Stipendio del 10%, restando sempre sotto 50000: la condizione resta vera per sempre, quindi T3 **non termina mai**

USO DEI TRIGGER

Controllo di coerenza dei dati

Trigger da definire: la relazione Confronto è asimmetrica. Dopo l'inserimento della tupla “A migliore di B su S”, verifica se esiste già la tupla simmetrica “B migliore di A su S”: se esiste, elimina la tupla appena inserita.

Superficie

Attributo	Chiave
CodSuperficie	**PK**
Nome	—

Confronto

Attributo	Chiave
CodSuperficie	**PK, FK**
Migliore	**PK, FK**
Peggiore	**PK, FK**

Tennista

Attributo	Chiave
CodTennista	**PK**
Nome	—

Migliore e **Peggiore** sono entrambe FK verso Tennista: una FK da sola non può impedire che esistano insieme (S,A,B) e (S,B,A) — serve il trigger

TRIGGER E CONTROLLO DI COERENZA

Un'istanza che viola il vincolo

Lo schema da solo non impedisce questa coppia di righe nella tabella `Confronto`:

Superficie	Migliore	Peggiore
Erba	**Sinner**	**Alcaraz**
Erba	**Alcaraz**	**Sinner**

Nessun vincolo dichiarativo (chiave primaria, FK, CHECK) può impedirlo: **un CHECK non può confrontare una riga con le altre righe della stessa tabella.** Serve un trigger che, prima o dopo ogni insert, verifichi se la riga simmetrica esiste già.

USO DEI TRIGGER

Controllo di coerenza dei dati

Trigger da definire: il direttore di un dipartimento deve afferirvi. Dopo l'inserimento o la modifica di “il Dip. X ha come direttore l'impiegato Y”, verifica se Y afferisce a X: se non è così, correggi l'incoerenza.

`Impiegato`

Attributo	Chiave
`ImpNum`	**PK**
`Nome`	—
`CodDip`	**FK**

`Dipartimento`

Attributo	Chiave
`CodDip`	**PK**
`Nome`	—
`Direttore`	**FK**

`Direttore` e **`CodDip`** sono due FK su tabelle diverse: nessuna FK può imporre che l'Impiegato Direttore afferisca proprio a quel Dipartimento — serve il trigger

TRIGGER E CONTROLLO DI COERENZA

Un'istanza che viola il vincolo

Il Dipartimento D1 ha come direttore I7, ma I7 afferisce a un altro dipartimento:

Dipartimento

CodDip	Nome	Direttore
D1	Vendite	**I7**

Impiegato

ImpNum	Nome	CodDip
I7	Marco Neri	**D3**

I7 dirige D1 ma afferisce a D3: **nessuna FK impedisce questa incoerenza** — serve un trigger che la rilevi e la corregga

USO DEI TRIGGER

Controllo di coerenza dei dati

Trigger da definire: dopo l'inserimento o la modifica di Frequenza(Stud1, C1), individua il professore del corso C1 (es. Prof1) e verifica se la coppia (Stud1, Prof1) esiste già in Docenza: se manca, inseriscila

Corso

Attributo	Chiave
CodCorso	PK
Nome	—
CodProf	FK

Frequenza

Attributo	Chiave
Studente	PK, FK
Corso	PK, FK

Docenza

Attributo	Chiave
Matricola	PK, FK
CodProf	PK, FK

Docenza deve contenere esattamente le coppie derivabili da Frequenza e Corso: è un dato derivato — serve il trigger per tenerlo sincronizzato a ogni insert in Frequenza.

TRIGGER E CONTROLLO DI COERENZA

Un'istanza che viola il vincolo

Frequenza e Corso implicano che Prof1 insegna a Stud1, ma Docenza non lo riflette:

Corso

CodCorso	CodProf
C1	Prof1

Frequenza

Studente	Corso
Stud1	C1

Docenza

Matricola	CodProf
manca (Stud1, Prof1)	

Nessun vincolo dichiarativo può “calcolare” Docenza a partire dalle altre tabelle: **serve un trigger che la aggiorni.**

Un esempio concreto

- Nel DB abbiamo: **`Corso(C1, Prof1)`**
- Viene aggiunta una nuova tupla: **`Frequenza(Stud1, C1)`**

```
create trigger AggiornaDocenza
after insert on Frequenza
when (not exists (select * from Docenza where Matricola = New.Studente
  and CodProf = (select CodProf from Corso where cod_corso = New.Corso)))
  insert into Docenza values (New.Studente,
    (select CodProf from Corso where cod_corso = New.Corso))
```

Il trigger aggiunge la nuova tupla: **`Docenza(Prof1, Stud1)`**

Discussione

A cosa servono i trigger? Posso fare tutto da applicazione!

- I trigger servono per implementare la logica dei dati, non delle applicazioni
- Permettono di implementare caratteristiche “invarianti” dei dati, che devono valere qualunque applicazione acceda ai dati

Altri vantaggi

- Non possono essere aggirati: anche uno script, un'altra applicazione o un accesso diretto al DB rispettano comunque il vincolo
- Permettono di esprimere vincoli che lo schema SQL dichiarativo non può catturare, come visto con Confronto, Impiegato/Dipartimento e Docenza
- Possono mantenere automaticamente sincronizzati dati derivati, senza che nessuna applicazione debba occuparsene esplicitamente
- Sono utili per compiti trasversali come audit e logging: registrare automaticamente chi e quando modifica i dati

Limiti e problemi

Ma i trigger hanno anche un costo:

- La logica è “nascosta”: leggendo solo il codice applicativo non si vede cosa succede davvero sui dati, rendendo il sistema più difficile da capire e da debuggare
- Gli effetti si propagano a cascata e possono essere difficili da prevedere, soprattutto con più trigger in conflitto sullo stesso evento
- Rischio di non terminazione: un ciclo nel grafo di triggering può non terminare mai, come visto con T3
- Un trigger before che controlla e poi scrive può essere scavalcato da una transazione concorrente, se l'isolamento non è sufficiente

Limiti pratici

- Ogni scrittura può scatenare query aggiuntive: su operazioni massive (bulk insert) l'overhead introdotto dai trigger può diventare significativo
- La sintassi varia molto da un DBMS all'altro (Oracle, PostgreSQL, SQL Server, ...): il codice dei trigger è spesso poco portabile
- Sono difficili da testare e mantenere

In sintesi: i trigger sono utili per invarianti semplici e locali ai dati — vanno usati con misura, non come sostituto della logica applicativa

BASI DI DATI

Autorizzazioni e controllo degli accessi

AUTORIZZAZIONI

Controllo dell'accesso

In SQL è possibile specificare **chi** (utente) e **come** (lettura, scrittura, ...) può utilizzare la base di dati (o parte di essa)

AUTORIZZAZIONI

Controllo dell'accesso

- Oggetto dei privilegi sono di solito le tabelle, ma anche altri tipi di risorse: singoli attributi, viste o domini
- Un utente predefinito (amministratore della base di dati) ha tutti i privilegi
- Il creatore di una risorsa ha tutti i privilegi su di essa

AUTORIZZAZIONI

Privilegi

Un privilegio è caratterizzato da:

- La risorsa cui si riferisce
- L'utente che concede il privilegio
- L'utente che riceve il privilegio
- L'azione che viene permessa
- La trasmissibilità del privilegio

AUTORIZZAZIONI

Tipi di privilegi offerti da SQL

- `insert`: permette di inserire nuovi oggetti (ennuple)
- `update`: permette di modificare il contenuto
- `delete`: permette di eliminare oggetti
- `select`: permette di leggere la risorsa
- `references`: permette vincoli di integrità referenziale verso la risorsa (può limitare la modifica da parte del proprietario, es. con “no action”)
- `alter`: permette modifiche allo schema

GRANT E REVOKE

Concessione di privilegi

```
grant <Privileges | all privileges>
  on Resource
  to Users [with grant option]
```

grant option specifica se il privilegio può essere trasmesso ad altri utenti

```
grant select on Department to Stefano with grant option
```

GRANT E REVOKE

Revoca di privilegi

```
revoke Privileges on Resource from Users [restrict | cascade]
```

- `restrict`: se gli Users hanno concesso i privilegi ad altri, blocca la revoca
- `cascade`: se gli Users hanno concesso i privilegi ad altri, propaga la revoca in cascata

GRANT E REVOKE

Esempi

```
GRANT SELECT ON films TO PUBLIC;
REVOKE INSERT ON films FROM PUBLIC;

GRANT SELECT, UPDATE ON mytable TO bill;

GRANT SELECT(title), UPDATE(title) ON films TO alice;

GRANT ALL PRIVILEGES ON films TO bill;
```

PUBLIC = tutti gli utenti

AUTORIZZAZIONI

Segretezza

La gestione delle autorizzazioni deve “nascondere” gli elementi cui un utente non può accedere, senza sospetti

- `Impiegati` non esiste
- `ImpiegatiSegreti` esiste, ma l'utente non è autorizzato

L'utente deve ricevere lo **stesso messaggio** in entrambi i casi

AUTORIZZAZIONI

Visibilità

Come autorizzare un utente a vedere solo alcune tuple di una relazione?

Attraverso una vista:

- Definiamo la vista con una condizione di selezione
- Attribuiamo le autorizzazioni sulla vista, anziché sulla relazione di base

AUTORIZZAZIONI

Autorizzazioni e ruoli

Concetto di **ruolo**, cui si associano privilegi (anche articolati), poi concessi agli utenti attribuendo il ruolo

Attribuisci la membership al ruolo “amministratori” allo user “joe”:

```
GRANT SELECT on films TO amministratori;
GRANT amministratori TO joe;
```

AUTORIZZAZIONI

Creazione utenti e ruoli

Creo un nuovo utente:

```
CREATE USER jim WITH PASSWORD 'jw8s0F4';
```

Creo un nuovo ruolo:

```
CREATE ROLE amministratori;
```